

Relatório de Análise de Segurança de Rede

Módulo 1: Sniffing e Interceptação (Camada 2 e 3)

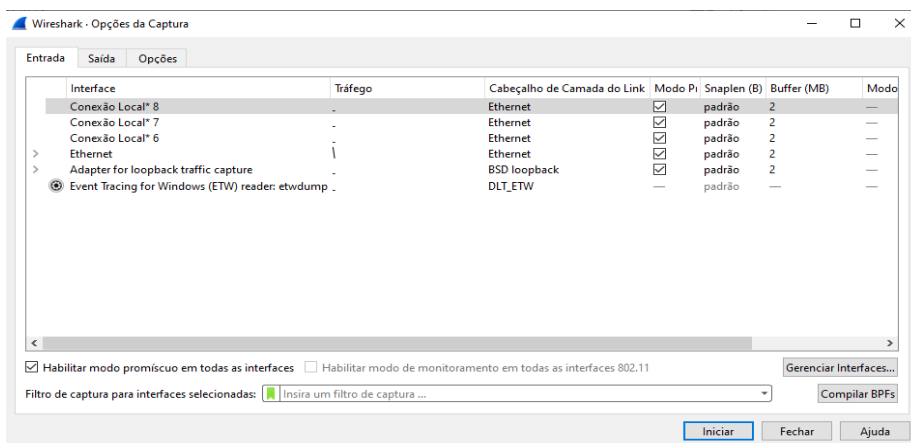
Ferramenta Utilizada: Wireshark

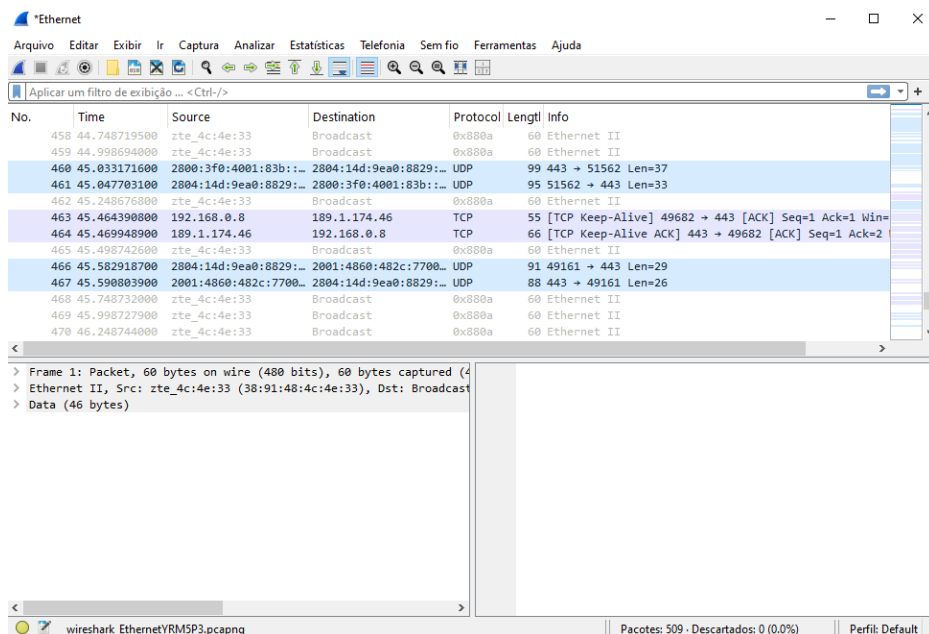
Passo a Passo da Captura:

1. **Inicialização:** O software Wireshark é iniciado.
2. **Configuração do Modo Promísquo:** Navega-se até o menu **Captura** e, em **Opções**, verifica-se e assegura-se a ativação do **modo promísquo** em todas as interfaces de rede. Este passo é crucial para a interceptação de tráfego não diretamente endereçado à máquina do analista.
3. **Seleção da Interface:** A interface de rede ativa (e.g., Ethernet, Wi-Fi) que apresenta tráfego de dados é selecionada.
4. **Execução e Análise:** A captura em tempo real é executada por alguns segundos para coletar pacotes, e o processo é interrompido para análise visual.

Resultado e Explicação Técnica:

O **Wireshark** capturou o tráfego da rede (protocolos, IPs e dados) usando o **modo promísquo**. Normalmente, a placa de rede só aceita pacotes destinados a ela, mas nesse modo passa a capturar todos os pacotes da rede, permitindo interceptar comunicações de outros dispositivos.





Módulo 2: Criptografia e Handshake (A base do TLS)

Ferramentas Utilizadas: Navegador Web e Ferramentas de Programador (DevTools)

Passo a Passo da Análise:

- Acesso a Site HTTPS:** Um site protegido pelo protocolo HTTPS é acessado.
- Inspecção do Certificado:** O certificado de segurança do servidor é inspecionado manualmente através do ícone de cadeado/configurações na barra de endereços (URL).
- Validação com DevTools:** O painel de segurança (aba Security) nas Ferramentas de Desenvolvedor (F12) é aberto para validar os parâmetros técnicos da conexão criptografada.

Resultado e Explicação Técnica:

A análise mostra que a conexão usa **criptografia híbrida**, combinando algoritmos assimétricos (como RSA/ECC) para a troca inicial de chaves e simétricos (como AES)

para criptografar os dados. Isso é a base do HTTPS.

Criptografia Híbrida e o Handshake TLS:

A criptografia assimétrica é mais lenta, então é usada apenas no início, durante o **handshake TLS**, para trocar com segurança uma chave de sessão. Depois disso, a comunicação passa a usar criptografia simétrica, que é mais rápida.

O handshake TLS envolve várias etapas cruciais:

1. **Client Hello:** Cliente envia suas opções de segurança (Client Hello).
2. **Server Hello:** Servidor responde com configurações e certificado (Server Hello).
3. **Verificação do Certificado:** Cliente verifica o certificado.
4. **Troca de Chaves:** O cliente envia um segredo criptografado, usado para gerar a chave de sessão.
5. **Finalização:** Ambos confirmam a conexão segura e iniciam a comunicação criptografada.

Visualizador do certificado: *.google.com

Geral

Detalhes

Emitido para

Nome comum (CN)	*.google.com
O (Organização)	<Não faz parte do certificado>
Unidade organizacional (OU)	<Não faz parte do certificado>

Emitido por

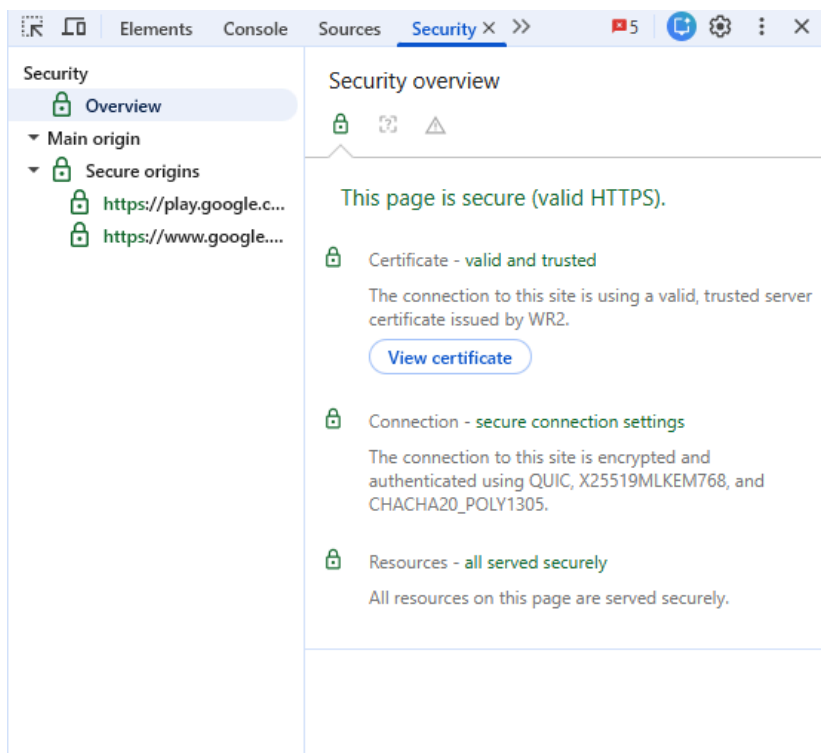
Nome comum (CN)	WR2
O (Organização)	Google Trust Services
Unidade organizacional (OU)	<Não faz parte do certificado>

Período de validade

Emitido em	quinta-feira, 7 de maio de 2026 às 12:51:26
Expira em	quinta-feira, 30 de julho de 2026 às 12:51:25

Impressões digitais SHA-256

Certificado	16a1eb63d1fcc4780095ec361205639eccabad6d2ef7aa6d6f6d175bceb9a2c
Chave pública	f0328953ffd415c3638843e136ab1743571e7b0baa82aed173597e8fd69f4f1



Módulo 3: Segurança na Camada de Aplicação (Desenvolvimento)

Ferramenta Utilizada: Ferramentas de Desenvolvedor do Navegador Web (Aba Application / Aplicativo)

Passo a Passo da Análise:

1. **Autenticação:** Autenticação (login) em uma aplicação web (ex: GitHub).
2. **Abertura do DevTools:** Abertura das Ferramentas de Desenvolvedor (F12) do navegador.
3. **Navegação à Aba 'Application':** Navegação até a aba Application (Aplicativo) e expansão da seção de Cookies no painel lateral.
4. **Identificação do Cookie de Sessão:** Identificação do cookie responsável por manter a sessão do usuário ativa.

Resultado e Explicação Técnica:

Foi possível identificar o valor do **cookie SessionID**, usado para manter o usuário logado.

Como o **HTTP** é sem estado, esse cookie funciona como um identificador único enviado em cada requisição, permitindo ao servidor reconhecer a sessão.

O Protocolo HTTP e o Cookie SessionID:

O **HTTP** é sem estado, ou seja, não guarda informações entre requisições. Para manter o usuário logado, utiliza-se o **Cookie SessionID**, que identifica a sessão e é enviado em cada requisição, permitindo ao servidor reconhecer o usuário e manter seu estado.

Prevenção de Session Hijacking:

O **Session Hijacking** ocorre quando um invasor intercepta o SessionID (principalmente em conexões HTTP) e consegue acessar a conta da vítima sem precisar da senha, se passando por ela. Para evitar isso, é fundamental usar **HTTPS**, que criptografa os dados, além de aplicar outras medidas de segurança para proteger a sessão.

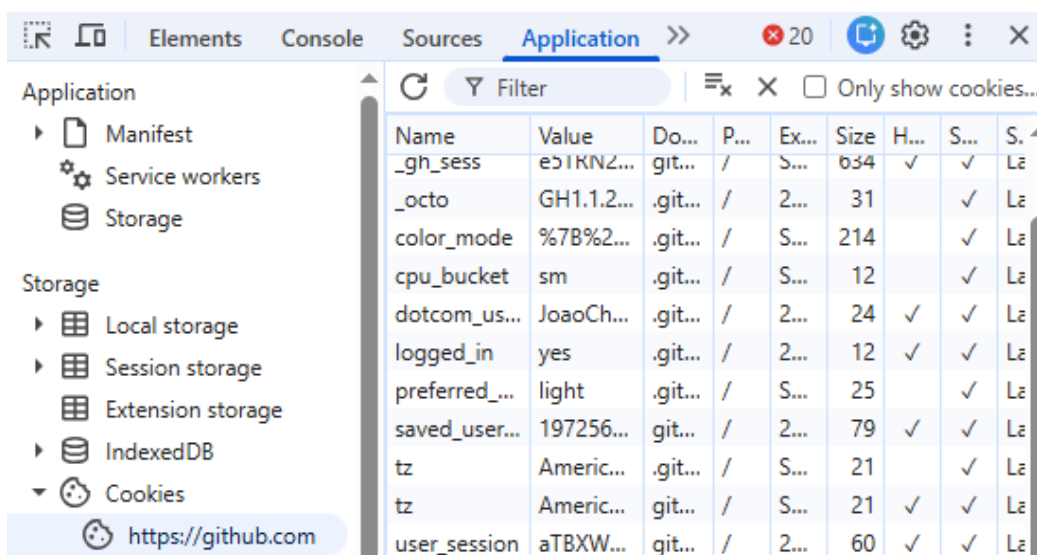
HttpOnly Flag: Impede que JavaScript acesse o cookie, ajudando a prevenir ataques XSS que tentam roubá-lo.

Secure Flag: Garante que o cookie seja enviado apenas via HTTPS, protegendo contra interceptação em redes inseguras.

SameSite Attribute: Ajuda a prevenir CSRF, controlando quando os cookies são enviados em requisições de terceiros.

Tempo de Expiração: Define um tempo de expiração para o cookie, exigindo novo login após inatividade e reduzindo riscos de ataque.

Rotação de SessionID: Alterar o SessionID após eventos críticos, como login ou mudança de senha, para invalidar sessões antigas e dificultar o sequestro.



The screenshot shows the Chrome DevTools Application tab with the 'Application' panel selected. The left sidebar shows the 'Storage' section expanded, with 'Cookies' selected for the URL 'https://github.com'. The main panel displays a table of cookies.

Name	Value	Domain	Path	Expires	Size	HttpOnly	Secure	SameSite
_gh_sess	e51KNZ...	git...	/	S...	634	✓	✓	L2
_octo	GH1.1.2...	.git...	/	2...	31		✓	L2
color_mode	%7B%2...	.git...	/	S...	214		✓	L2
cpu_bucket	sm	.git...	/	S...	12		✓	L2
dotcom_us...	JoaoCh...	.git...	/	2...	24	✓	✓	L2
logged_in	yes	.git...	/	2...	12	✓	✓	L2
preferred_...	light	.git...	/	S...	25		✓	L2
saved_user...	197256...	git...	/	2...	79	✓	✓	L2
tz	Americ...	.git...	/	S...	21		✓	L2
tz	Americ...	git...	/	S...	21	✓	✓	L2
user_session	aTBXW...	git...	/	2...	60	✓	✓	L2